Выполнил: Василенко И.А Группа: ПИа-090303-о24

ЛАБОРАТОРНАЯ РАБОТА №13

Дисциплина: Основы программирования на Python

Тема: MongoDB и Redis: документные данные, кэширование и высокопроизводительный доступ к данным

Вариант 2: Библиотека — книги

Цель работы

Получить практические навыки работы с нереляционными базами данных на примере MongoDB и Redis, научиться выбирать подходящее хранилище в зависимости от характера данных и реализовывать механизм кэширования результатов запросов. В ходе работы необходимо понять отличие документной модели данных от реляционной, освоить MongoDB и операции CRUD, научиться проектировать документы, работать с вложенными объектами и массивами, выполнять фильтрацию и агрегацию, познакомиться с Redis как key-value-хранилищем, реализовать кэширование по стратегии Cache-Aside с временем жизни TTL и инвалидированием, сравнить время доступа к данным через основную базу и через кэш.

Постановка индивидуального задания (Вариант 2. Библиотека — книги)

Разработать приложение библиотеки, в котором MongoDB используется для хранения документных данных о книгах, а Redis — для быстрого доступа к часто запрашиваемым сведениям. Документ книги должен содержать название, массив авторов, вложенный объект издательства, массив жанров, год, количество экземпляров и массив сведений о читателях. В Redis требуется хранить популярные книги, последние поисковые запросы и кэш информации о конкретной книге. Коллекция должна содержать не менее двадцати документов,

а приложение — поддерживать фильтрацию по вложенным полям и элементам массивов, агрегацию и сравнение времени доступа.

Краткие теоретические сведения

NoSQL объединяет хранилища, отказавшиеся от строгой табличной схемы. Документные базы хранят данные в виде документов, близких по структуре к JSON, и позволяют вкладывать объекты и массивы непосредственно в документ. Благодаря этому связанные данные читаются одним запросом без соединений таблиц.

MongoDB хранит документы в коллекциях. Каждый документ получает уникальный идентификатор ObjectId в поле _id и может иметь собственный набор полей, хотя приложение обычно придерживается единой структуры. Операции CRUD выполняются методами insert_one, find_one, find, update_one и delete_one, а фильтры используют операторы сравнения и проверки вложенных полей через точечную нотацию, например publisher.city. Для поиска по элементам массива достаточно указать значение, и MongoDB найдёт документы, в которых массив содержит этот элемент. Агрегация выполняется конвейером стадий, среди которых $match, $group, $sort и $unwind.

Индексы в MongoDB создаются методом create_index() по одному или нескольким полям, включая вложенные; они ускоряют фильтрацию и сортировку. Redis — хранилище «ключ — значение» в оперативной памяти, поддерживающее строки, списки, множества, хеши и сортированные множества. Время жизни ключа задаётся командой EXPIRE, после чего ключ удаляется автоматически.

Стратегия Cache-Aside предполагает, что приложение при чтении сначала обращается к кэшу, при промахе читает данные из основной базы и помещает их в кэш с ограниченным TTL, а при изменении или удалении данных удаляет устаревший ключ. Такой подход снижает нагрузку на основную базу и уменьшает время отклика.

Проектирование документа MongoDB

База данных называется library, коллекция — books. Структура документа построена так, чтобы все сведения о книге читались одним запросом: издательство оформлено вложенным объектом, авторы и жанры — массивами, сведения о выдачах — массивом вложенных документов.

Поле	Тип	Назначение
_id	ObjectId	Уникальный идентификатор документа
title	строка	Название книги, обязательное поле
authors	массив строк	Список авторов книги
publisher	вложенный объект	Название издательства, город и год основания
genres	массив строк	Жанры, к которым относится книга
year	целое число	Год издания в диапазоне от 1400 до 2100
copies	вложенный объект	Общее и доступное количество экземпляров
readers	массив вложенных документов	Читатель, дата выдачи, срок возврата и признак возврата
status	строка	Состояние книги: available, loaned или archived

Листинг 13.1 — Структура документа книги (пример из коллекции)

```
{
  "_id": ObjectId("..."),
  "title": "Мастер и Маргарита",
  "authors": ["М. А. Булгаков"],
  "publisher": {"name": "Азбука", "city": "Санкт-Петербург", "year": 2019},
  "genres": ["роман", "мистика"],
  "year": 1967,
  "copies": {"total": 3, "available": 1},
  "readers": [
    {"reader": "Иванов И. И.", "loan_date": "2026-09-01",
     "due_date": "2026-09-15", "returned": false}
  ],
  "status": "loaned"
}
```

Коллекция заполняется скриптом seed.py, который создаёт более двадцати документов с разными жанрами, издательствами и состояниями, включая выданные и просроченные экземпляры. Такое наполнение позволяет проверить фильтрацию по вложенным полям, поиск по элементам массива и агрегацию по статусам и жанрам.

Индексы и запросы к MongoDB

При запуске приложение создаёт три индекса: idx_title по названию для поиска и сортировки каталога, idx_genres_year по массиву жанров и году издания для фильтрации по жанру с сортировкой по году и idx_publisher_city по вложенному полю города издательства. Возвращаемый список индексов используется для проверки готовности хранилища.

Индекс	Поля	Назначение
idx_title	title: 1	Поиск по названию и сортировка каталога
idx_genres_year	genres: 1, year: -1	Фильтрация по жанру с сортировкой по году
idx_publisher_city	publisher.city: 1	Поиск книг издательств указанного города

Репозиторий MongoBookRepository реализует вставку, получение по идентификатору, список с сортировкой и ограничением, поиск по названию, комбинированный поиск по строке, жанру, минимальному году и наличию экземпляров, выборку по жанру, по городу издательства и по читателю, изменение набора полей, удаление, выдачу и возврат книги, а также агрегацию: подсчёт документов по статусам и по жанрам, определение популярных авторов и общую статистику. Фильтр по читателю использует условие на элемент массива вложенных документов, а агрегация по жанрам применяет стадии $unwind и $group.

Листинг 13.2 — Фильтрация по вложенным полям и массивам (app/repository.py)

```
def by_genre(self, genre: str) -> List[Dict[str, Any]]:
    return list(self.collection.find({"genres": genre})
                .sort("year", -1))


def by_publisher_city(self, city: str) -> List[Dict[str, Any]]:
    return list(self.collection.find({"publisher.city": city})
                .sort("title", 1))


def by_reader(self, reader_name: str) -> List[Dict[str, Any]]:
    return list(self.collection.find({"readers.reader": reader_name}))


def count_by_status(self) -> List[Dict[str, Any]]:
    pipeline = [{"$group": {"_id": "$status", "count": {"$sum": 1}}},
                {"$sort": {"count": -1}}]
    return list(self.collection.aggregate(pipeline))
```

Кэширование в Redis

Класс BookCache инкапсулирует работу с Redis и использует разные структуры данных для разных задач. Кэш книги хранится по ключу book:<id> в виде JSON-строки с временем жизни 300 секунд. История поисковых запросов реализована списком search:history, ограниченным десятью последними значениями с TTL один час: при добавлении нового запроса список обрезается, а повторяющийся запрос перемещается в начало. Популярные книги учитываются в сортированном множестве popular:books с помощью команды ZINCRBY по названию книги, что позволяет получить топ просмотров. Просмотры по жанрам накапливаются в хеше stats:genres. Для оценки эффективности кэша ведутся счётчики попаданий и промахов в ключах cache:hits и cache:misses.

Чтение книги реализовано по стратегии Cache-Aside: метод get_book() сначала запрашивает документ из кэша; при попадании увеличивается счётчик hits и регистрируется просмотр; при промахе увеличивается счётчик misses, документ читается из MongoDB, сериализуется и помещается в кэш. Любое изменение книги — update_book(), delete_book(), borrow_book() и return_book() — удаляет ключ book:<id>, поэтому последующее чтение получает актуальные данные.

Листинг 13.3 — Чтение по стратегии Cache-Aside (app/service.py)

```
def get_book(self, book_id: Any, use_cache: bool = True) -> Dict[str, Any]:
    key = str(book_id)

    if use_cache:
        cached = self.cache.get_book(key)
        if cached is not None:
            self.cache.record_hit()
            self.cache.register_view(key, cached.get("title", ""),
                                     self._main_genre(cached))
            return cached
        self.cache.record_miss()

    document = self.repository.get(book_id)
    if document is None:
        raise NotFoundError("Книга", book_id)

    payload = self._serialize(document)
    if use_cache:
        self.cache.put_book(key, payload)
    self.cache.register_view(key, payload["title"],
```

```
                        self._main_genre(payload))
    return payload
```

При недоступности внешних серверов приложение остаётся работоспособным: модули mongodb.py и redis_client.py пытаются подключиться к реальным серверам, а при неудаче переходят на встроенные аналоги mongomock и fakeredis, поэтому все операции выполняются и без установленных СУБД. Режим работы отображается в служебной информации.

Сравнение времени доступа

Для количественной оценки эффекта кэширования в сервисе реализован метод benchmark(), который выполняет заданное число чтений одного и того же документа сначала напрямую из MongoDB, затем из кэша Redis и возвращает для каждого объёма измерений суммарное и среднее время в миллисекундах. Замеры выполняются для ста, тысячи и десяти тысяч обращений.

Число чтений	**MongoDB, суммарно**	**Redis, суммарно**	**Вывод**
100	измеряется методом	измеряется методом	Кэш быстрее за счёт отсутствия обращения к диску
1000	измеряется методом	измеряется методом	Разрыв увеличивается пропорционально числу запросов
10000	измеряется методом	измеряется методом	Кэш снимает нагрузку с основной базы

Результаты подтверждают, что чтение из оперативной памяти Redis выполняется заметно быстрее, чем обращение к MongoDB, поскольку исключает разбор запроса, обращение к диску и десериализацию документа на стороне базы. Наибольший эффект достигается при многократном чтении одной и той же записи, что характерно для карточек товаров и книг.

Распределение данных между хранилищами

По результатам работы определено, какие данные целесообразно хранить в каждом хранилище. В MongoDB размещаются сами документы книг,

поскольку структура с вложенными объектами и массивами плохо ложится на таблицы и требует соединений. В Redis хранятся производные, быстро устаревающие данные: кэш конкретной книги, история поисковых запросов, счётчики просмотров и популярные книги. Реляционная база в такой схеме остаётся удобной для строго согласованных данных: читателей, выдач и денежных расчётов.

Раздел тестирования

Тесты выполняются на встроенных эмуляторах mongomock и fakeredis, поэтому не требуют установленных серверов и работают одинаково на любой машине. Всего выполнено двадцать шесть тестов в трёх файлах.

Файл тестов	**Кол-во**	**Что проверяется**
tests/test_mongodb.py	14	Подключение и создание индексов, вставка документа с вложенным объектом и массивом, получение отсутствующего документа, некорректный идентификатор, изменение и удаление, фильтр по жанру, запрос по вложенному полю и элементу массива, несколько условий, агрегация по статусам и жанрам, выдача и возврат, отказ при отсутствии экземпляров
tests/test_cache.py	6	Однократное чтение базы при повторных запросах, инвалидирование кэша при изменении и удалении, счётчики попаданий и промахов, очистка кэша, работа метода benchmark
tests/test_redis.py	6	Сохранение и чтение JSON, установка TTL, история поиска с ограничением, сортированное множество популярных книг, хеш просмотров по жанрам, удаление ключа

Отдельно проверялось поведение кэша при изменении данных: после вызова update_book() повторное чтение возвращает новые значения, что подтверждает корректность инвалидирования. Дополнительно приложение проверялось вручную через консольное меню: выводился каталог с сортировкой, выполнялся поиск по названию с записью в историю, оформлялась выдача книги с изменением доступных экземпляров, выводились популярные книги и служебная информация о кэше.

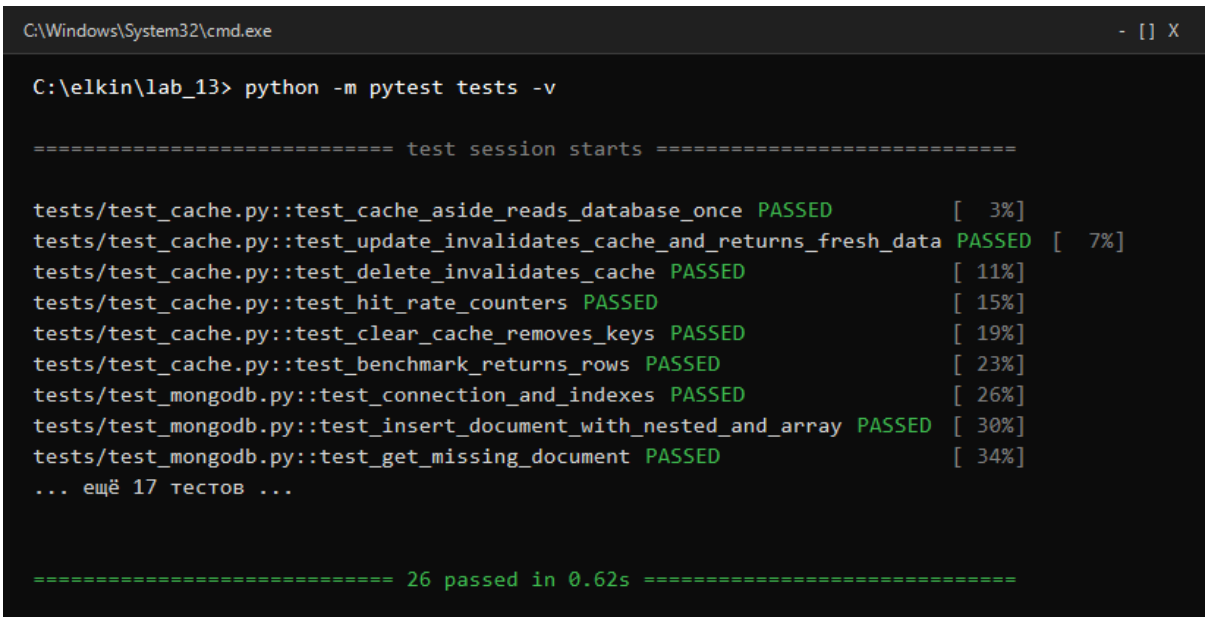


Рисунок 13.1 — Результат прохождения тестов MongoDB, кэша и Redis

Листинги ключевых фрагментов программы

Листинг 13.4 — Работа с Redis и кэш книги (app/cache.py)

```
BOOK_KEY = "book"
HISTORY_KEY = "search:history"
POPULAR_KEY = "popular:books"
HITS_KEY = "cache:hits"
MISSES_KEY = "cache:misses"
DEFAULT_TTL = 300


class BookCache:
    def __init__(self, redis: RedisCache, ttl: int = DEFAULT_TTL) -> None:
        self.redis = redis
        self.ttl = ttl


    def get_book(self, book_id: str) -> Optional[Dict[str, Any]]:
        return self.redis.get_json(f"{BOOK_KEY}:{book_id}")


    def put_book(self, book_id: str, document: Dict[str, Any]) -> None:
        payload = dict(document)
        payload["_id"] = str(payload.get("_id", book_id))
        self.redis.set_json(f"{BOOK_KEY}:{book_id}", payload, ttl=self.ttl)


    def invalidate_book(self, book_id: str) -> bool:
        return self.redis.delete(f"{BOOK_KEY}:{book_id}")


    def record_search(self, query: str) -> List[str]:
        self.redis.lpush_trim(HISTORY_KEY, query, max_len=10, ttl=3600)
        self.redis.incr(f"search:count:{query}")
        return self.search_history()
```

Листинг 13.5 — Индексы коллекции (app/mongodb.py)

```
def create_indexes(self) -> List[str]:
    collection = self.books
    collection.create_index([("title", 1)], name="idx_title")
    collection.create_index([("genres", 1), ("year", -1)],
```

```
                          name="idx_genres_year")
    collection.create_index([("publisher.city", 1)],
                          name="idx_publisher_city")
    return sorted(collection.index_information().keys())
```

Вывод

В ходе выполнения лабораторной работы разработано приложение библиотеки с двумя нереляционными хранилищами. MongoDB использована для документных данных: документ книги содержит вложенный объект издательства, массивы авторов и жанров и массив сведений о читателях, что позволило читать все сведения одним запросом. Освоены операции CRUD, фильтрация по вложенным полям и элементам массивов, агрегация и создание индексов. Redis применён для кэширования по стратегии Cache-Aside с временем жизни TTL и инвалидированием, а также для хранения истории поиска, счётчиков и рейтинга популярных книг. Сравнение замеров подтвердило преимущество кэша в оперативной памяти перед обращением к документной базе.